# 06 — Komplexe Datentypen

Listen, Tupel, Sets, Dictionaries

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

# Agenda

1. Wozu Sammlungen? – viele Werte, ein Name
2. **Listen** – geordnet & veränderbar
3. **Tupel** – geordnet & eingefroren
4. **Sets** – Mengen ohne Duplikate
5. **Dictionaries** – Schlüssel $\longrightarrow$ Wert
6. Welche Sammlung wann?
7. **Klassiker: Einkaufsliste** – live programmiert

**Lernziel**: Warenkorb, Katalog und Bestellungen von Veggie Soles mit dem **passenden** Sammlungstyp modellieren – und verstehen, **wie** Python ein Element wiederfindet.

**Heute**: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 06).

# Wozu Sammlungen?

## Ohne Sammlung

```
produkt_1 = "Eco-Sneaker"
produkt_2 = "Hemp-High"
produkt_3 = "Bambus-Boot"    # und bei 50 Produkten?
```

## Mit Sammlung

```
produkte = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]
```

**Sammlung** = ein Name für **mehrere Werte**. Ein Warenkorb hat mal 2, mal 8 Artikel – eine Variable pro Artikel kann das nicht abbilden. Python bietet **vier** Sammlungstypen, jeder mit einer eigenen Stärke.

# Die vier Sammlungstypen im Überblick

| Typ | Klammer | Geordnet | Veränderbar | Duplikate |
|---|---|---|---|---|
| **Liste** | `[ ]` | ja | ja | ja |
| **Tupel** | `( )` | ja | – | ja |
| **Set** | `{ }` | – | ja | – |
| **Dict** | `{k: v}` | ja | ja | Schlüssel: nein |

```
liste = ["Eco-Sneaker", "Hemp-High"]
tupel = ("Eco-Sneaker", 89.95, 1001)
menge = {38, 39, 40, 41}
karte = {"Eco-Sneaker": 89.95}
```

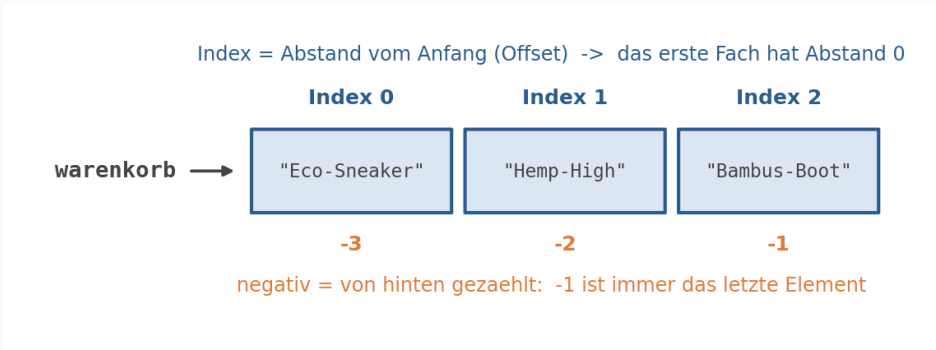
# Listen – geordnet & veränderbar

```
warenkorb = ["Eco-Sneaker", "Hemp-High",
             "Bambus-Boot"]

print(warenkorb[0])      # Eco-Sneaker
print(warenkorb[-1])     # Bambus-Boot
print(len(warenkorb))    # 3
```

- Eckige Klammern `[ ]`, Elemente durch Kommas getrennt
- Die **Reihenfolge bleibt erhalten** – Position 0, 1, 2, …
- Zugriff per **Index**: `warenkorb[0]` ist das **erste** Element
- `len(...)` liefert die Anzahl der Elemente

# Unter der Haube: Index = Abstand vom Anfang




Eine Liste ist eine **nummerierte Fächerreihe** im Speicher (E4: Adressen!): Python merkt sich den **Anfang** und springt von dort Index Fächer weiter. Das erste Fach liegt 0 Schritte vom Anfang – **deshalb** startet die Zählung bei 0. Negativ heißt von hinten: `liste[-1]` = `liste[len(liste)-1]`.

# Listen-Methoden: den Warenkorb pflegen

```
warenkorb.append("Bambus-Boot")    # hinten anhaengen
warenkorb.remove("Hemp-High")      # Wert entfernen
warenkorb[0] = "Cork-Slipper"      # ueberschreiben
```

- `append(x)` hängt **hinten** an – die Liste wächst
- `remove(x)` entfernt den **ersten Treffer** mit diesem **Wert** (nicht Index!) – der Rest rückt nach
- Zuweisung an einen Index **überschreibt** das Fach
- Methoden-Syntax `objekt.methode(...)` kennen Sie schon von den String-Methoden (NB 05)

# Unter der Haube: Mutabilität – dasselbe Objekt ändert sich

```
warenkorb = ["Eco-Sneaker", "Hemp-High"]  # Zeile 1
warenkorb.remove("Hemp-High")             # Zeile 2
warenkorb.append("Bambus-Boot")           # Zeile 3
```

| nach Zeile | `warenkorb` ist jetzt |
|---|---|
| 1 | `['Eco-Sneaker', 'Hemp-High']` |
| 2 | `['Eco-Sneaker']` |
| 3 | `['Eco-Sneaker', 'Bambus-Boot']` |

Kein = in Zeile 2 und 3! Die Methoden verändern **dasselbe Objekt im Speicher** – die Liste ist **mutabel** (veränderbar). Es entsteht keine Kopie: alle drei Zeilen arbeiten am selben Warenkorb.

# Listen – Demo & Übung

▶ **Live-Demo** – `01_listen_basics.py`, `02_listen_operationen.py`

✎ **Sofort ausprobieren** – Notebook 06, Kap. 1: Liste `städte` anlegen und per Index zugreifen, `werkzeuge` mit `append`, `tiere` mit `remove`

# Tupel – die eingefrorene Liste

```
produkt = ("Eco-Sneaker", 89.95, 1001)
#          Name           Preis  ID

print(produkt[1])              # 89.95
# produkt[1] = 79.95           # TypeError!
name, preis, nr = produkt      # Entpacken
```

- Runde Klammern ( ), Index-Zugriff **wie bei Listen**
- Inhalt **steht fest**: kein append, kein Überschreiben
- **Tupel-Entpacken**: alle Werte auf einmal in Variablen

# Unter der Haube: warum einfrieren?

Absichtlich **weniger** können als die Liste – zwei Gründe:

- **Absicherung**: Stammdaten wie `(Name, Preis, ID)` sollen sich nicht *versehentlich* ändern. Python wirft lieber sofort `TypeError`, als still falsche Daten zu speichern
- **Verlässlichkeit**: nur was sich **nie** ändert, darf später als **Dict-Schlüssel** dienen (kommt gleich!)

| Frage | Typ | Veggie-Soles-Beispiel |
|---|---|---|
| ändert sich der Inhalt? | Liste | Warenkorb, Lagerbestand |
| steht der Inhalt fest? | Tupel | `("Eco-Sneaker", 89.95, 1001)` |

▶ **Live-Demo** – `03_tupel_immutable.py`

# Sets – Mengen ohne Duplikate

```
groessen = {40, 41, 40, 42, 41}
print(groessen)          # {40, 41, 42}

groessen.add(43)         # hinzufuegen
groessen.remove(40)      # entfernen
print(len(groessen))     # 3
```

- Geschweifte Klammern { }
- **Duplikate fliegen automatisch raus**
- **Keine Reihenfolge** – darum gibt es **kein** `groessen[0]`

▶ **Live-Demo** – `04_sets_unique.py`

# Unter der Haube: Set = Menge aus der Mathematik

Ein Set ist der **Mengenbegriff** aus der Mathematik, als Datentyp gegossen:

$$\{40,\ 41,\ 40,\ 42,\ 41\} \ = \ \{40,\ 41,\ 42\}$$

- Für eine Menge zählt nur: ein Wert **ist drin oder nicht** – wie oft er hineingelegt wurde, ist bedeutungslos
- Eine **Position** existiert nicht: $\{40, 41\}$ und $\{41, 40\}$ sind **dieselbe** Menge – deshalb verbietet Python den Index-Zugriff

✎ **Sofort ausprobieren** – Notebook 06, Kap. 3: doppelte Bestellgrößen mit `set(liste)` eindampfen, eindeutige Bauteile zählen

# Dictionaries – Schlüssel → Wert

```
preise = {"Eco-Sneaker": 89.95,
          "Hemp-High": 109.00}

print(preise["Hemp-High"])        # 109.0
preise["Bambus-Boot"] = 135.50    # neu anlegen
preise["Hemp-High"] = 99.00       # aendern
```

- Paare `schlüssel: wert` in geschweiften Klammern
- Zugriff über den **Schlüssel** – nicht über eine Position
- Jeder Schlüssel ist **eindeutig**; dieselbe Syntax `d[k] = v` legt neu an **oder** überschreibt

# Unter der Haube: Dict = Abbildung

Mathematisch ist ein Dict eine **Abbildung** $f$: Schlüssel $\to$ Wert – hier: $f(\text{Eco-Sneaker}) = 89.95$.

| Sammlung | Frage an sie | Antwort über |
|---|---|---|
| Liste | *Was steht an Position $k$?* | Offset vom Anfang |
| Dict | *Was gehört zu diesem Schlüssel?* | Rechnung aus dem Schlüssel |

Warum geht Nachschlagen "sofort"? Python **berechnet** aus dem Schlüssel direkt den Ablageort (Idee der *Hashtabelle*) – statt alle Fächer der Reihe nach abzusuchen. Deshalb müssen Schlüssel **unveränderlich** sein: `str`, `int`, Tupel – keine Listen!

# Dictionaries – Demo & Übung

▶ **Live-Demo** – `05_dict_produkt.py`

✎ **Sofort ausprobieren** – Notebook 06, Kap. 4: Telefonbuch-Dict anlegen, Werte ändern und ergänzen, `keys()` / `values()` ansehen

# Welche Sammlung wann?

| Situation | Typ | Syntax |
|---|---|---|
| Reihenfolge zählt, Inhalt ändert sich | **Liste** | `[ ]` |
| Inhalt steht ein für alle Mal fest | **Tupel** | `( )` |
| jeder Wert darf nur einmal vorkommen | **Set** | `{ }` |
| jeder Wert hat einen Namen | **Dict** | `{k: v}` |

**Faustregel**: im Zweifel eine **Liste**. Sobald Werte einen **Namen** haben (Produkt → Preis), ist das Dict die bessere Wahl.

# Klassiker: Einkaufsliste – die Aufgabe

Der Klassiker “Warenkorb pflegen” – nur mit dem Stoff von heute:

- Warenkorb als **Liste** mit 2 Produkten anlegen und ausgeben
- Nachbestellung: **Bambus-Boot** per append ergänzen
- Umtausch: **Hemp-High** per `remove` stornieren
- Preise als **Dictionary** (Produktname → Preis) hinterlegen
- **Gesamtpreis**: beide Positionen per Schlüssel nachschlagen und addieren

**So soll es laufen**

```
Warenkorb: ['Eco-Sneaker', 'Hemp-High']
Nach Umtausch: ['Eco-Sneaker', 'Bambus-Boot']
Gesamt: 225.45 EUR
```

# Klassiker: Einkaufsliste – Live-Coding

**▶ Live-Coding ab leerem Editor**

Referenz: `90_klassiker_einkaufsliste.py` – Teilziele:

1. Warenkorb-Liste mit 2 Produkten anlegen
2. append (Nachbestellung) und `remove` (Umtausch)
3. Preis-Dictionary anlegen
4. Preis per Schlüssel nachschlagen: `preise[warenkorb[0]]`
5. Gesamtpreis = Position 0 + Position 1 (explizite Indizes)

✎ **Zum Selberlösen** – Handout *“Klassiker 06: Einkaufsliste”* (`Klassiker/06_Klassiker_Einkaufsliste.pdf`). Vertiefung: Notebook 06, Trainingsaufgaben 1 und 6.

# Cheat Card

| Aufgabe | Syntax |
|---|---|
| Liste anlegen / lesen | `l = [1, 2]` · `l[0]` · `l[-1]` |
| Liste ändern | `l.append(x)` · `l.remove(x)` · `l[0] = x` |
| Tupel (fest) | `t = (1, 2)` · `t[0]` |
| Tupel entpacken | `a, b = t` |
| Set (eindeutig) | `s = {1, 2}` · `s.add(x)` · `s.remove(x)` |
| Dict anlegen / lesen | `d = {"k": 1}` · `d["k"]` |
| Dict ändern | `d["k"] = 2` · `d.pop("k")` |
| Größe (alle vier) | `len(x)` |
| Klassiker heute | Liste + append/remove + Preis-Dict + Index |

**Faustregel**: Reihenfolge & änderbar → **Liste** · fest → **Tupel** · eindeutig → **Set** · Name → Wert → **Dict**.

# Ausblick: Notebook 07 – Funktionen

Unser Klassiker hat eine Schwachstelle:

```
gesamt = preise[warenkorb[0]] + preise[warenkorb[1]]
```

Bei 8 Artikeln: 8 Indizes von Hand – und nach jedem append anpassen. Zwei Auswege stehen an:

- **Nächste Woche (NB 07)**: eigene **Funktionen** – Logik wie "Gesamtpreis berechnen" bekommt einen Namen und wird wiederverwendbar
- **Später (NB 10)**: **Schleifen** – der Rechner übernimmt das Aufzählen der Positionen komplett

Sammlungen sind das Material – Funktionen und Schleifen werden die Werkzeuge dazu.

# Heute geübt

✓ Vier Sammlungstypen und ihre Stärken unterschieden
✓ Listen: Index = Offset vom Anfang, deshalb ab 0; negativ = von hinten
✓ Mutabilität: append/`remove` verändern **dasselbe** Objekt
✓ Tupel: eingefroren als Absicherung – und als Dict-Schlüssel tauglich
✓ Sets: der mathematische Mengenbegriff, keine Duplikate, keine Ordnung
✓ Dicts: Abbildung Schlüssel $\longrightarrow$ Wert, Nachschlagen statt Suchen
✓ **Klassiker Einkaufsliste** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Einkaufsliste** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 06
- Trainingsmaterial: gestufte Aufgaben quer durch alle vier Sammlungstypen